H-Verifier: Verifying Confidential System State with Delegated Sandboxes

Anyi Liu[1[0000−0002−6011−1998]] and Guangzhi Qu[1[0000−0003−4047−9514]]

Department of Computer Science and Engineering, Oakland University, MI 48309, USA
{anyiliu, gqu@oakland.edu}

Abstract. As a viable technology that supports ubiquitous, versatile, and cost-efficient IT solution for various applications, the wide adoption of the cloud computing is still hindered by the lack of online anomaly detection and poor accountability. Although cloud providers facilitate trusted computing with the support of hardware and software, to construct a trusted computing base (TCB) in a hostile environment with a minimal size is still non-trivial. It is desired to have a general solution that supports trusted computing regardless of the underlying hardware and software. In this paper, we present a system, namely *H-Verifier*, which verifies the states of the confidential system with cryptographic support. To ensure data confidentiality, H-Verifier leverages homomorphic encryption (HE), partitions the model of the monitored system, and stores the partitions distributively. To track the system states, the partitions are updated against the ciphertext via the standard HE operations. Since both the model and audit events are encrypted, H-Verifier does not require a TCB to be constructed on the computational nodes. H-Verifier overcomes the performance curse of HE and makes sure that only a limited number of HE operations are required online. H-Verifier is capable of performing trusted computing without relying on special hardware and thus can be widely deployed in various sandboxes and/or enclave. The experimental results demonstrate that H-Verifier can achieve reasonable performance for model initialization, updating, and verification for an online system in the cloud.

Keywords: Homomorphic encryption · Finite-state automaton · Sandbox.

1 Introduction

As a ubiquitous, versatile, and cost-efficient IT solution, cloud computing has been widely adopted as a feasible technology to facilitate various services and applications. While the cloud computing offers the appealing features, such as elasticity and low cost, it has also been abused by the adversaries. Extensive research has shown that adversaries took advantage of the appealing features of the cloud and have exaggerated the loss caused by the cloud-based intrusions [1], malicious services [2], [3], [4], and underground ecosystems [5], [6]. The recently

announced the Security, Trust & Assurance Registry (STAR) [7] by the Cloud Security Alliance explicitly defines three levels of audit certification: self-auditing, third-party auditing, and continuous, near real-time verification of security compliance, which articulates the importance of real-time verification and auditing. To counter the capability of the adversary and grantee the timely detection of any violation of security properties, the runtime auditing, system verification, and intrusion detection become urgent to cloud providers to ensure the security status in the cloud and to be responsive to the cybercrime and intrusions.

However, several challenges in the area hinder the real-time verification and auditing. First, it is difficult to standardize the techniques of secure audit log generation and real-time tenant's system status verification. To increase the cloud tenants' trust on the designated infrastructure and platform, cloud providers must ensure the security compliance with the support of trusted computing. To construct a trusted execution environment (TEE), hardware and software are used to construct a trusted computing base (TCB) in a hostile environment with a minimal size. Existing systems primarily focus on constructing the root of trust (RoT) with the trusted hardware or hypervisor [8], [9], [10], [11], [12]. Commodity technologies, such as Intel System Management Mode (SMM) [13], Trusted Execution Technology (TXT) [14], and Intel Software Guard Extensions (SGX) [15] are proprietary products and lack of the cross-platform guarantee. Second, although the technology, such as virtual machine introspection (VMI) [16], [17], [18], allows the virtual artifacts to be identified and collected from the self-provisioned VMs, the correctness of the technology largely relies on the trustworthiness of the hypervisor or the privileged VM. Moreover, to reconstruct the virtual artifact and verify the security properties of the monitor system from the low-level observations requires to bridge the semantic gap [19], which is non-trivial. Thus, a general solution is desired regardless of the underlying infrastructure and the supporting technologies to construct the TEE.

In this paper, we present an online system, namely *H-Verifier*, which is capable of identifying the status of a system through the confidential audit log with the aid of collaborative sandboxes[1]. To ensure the confidentiality of audit log, H-Verifier partitions the model of the monitored system and encrypts the model by leveraging the homomorphic encryption (HE) [20]. The partitions are securely updated on different computational nodes. The major innovative contribution H-Verifier is that it overcomes the performance curse of HE by only introducing a limited number of HE operations when updating the partitions. Since the computational nodes only keep the ciphertext of the model and data, no additional TCB is needed on the computational nodes, which allows it to perform trusted computing without relying on special hardware and can be widely deployed in various sandboxes, including virtual machine, container, and enclave. The experimental results demonstrate that H-Verifier is capable of identifying the status of the monitor system and achieving a reasonable overhead for model initialization, updating, and verification for an online system in the cloud.

[1] We use *sandbox* and *enclave* interchangeably throughout this paper.

The rest of the paper is organized as follows. First, we review the related work Section 2. Then, we describe the threat model in Section 3. After that, we present the system design and elaborate the methodology in Section 4 and the evaluation results Section 5. We also discuss the limitation of our current design and proposed the future work in Section 6. Finally, we conclude the paper in Section 7 .

2 Related Work

As a cryptosystem that allows the untrusted party to perform computing on the encrypted data, while still preserves ciphertext secret, the Homomorphic Encryption (HE) has been widely adopted in various applications [21], [22], and [23]. Li et al. [22] presented a distributed incremental data aggregation scheme, in which data collected from smart meters can be aggregated along the route from the source meter to the collector unit. Homomorphic encryption is used to protect user's private data en route. As a result, the involved meters cannot see any intermediate or final result. Hong et al. [23] proposed a collaborative search log sanitization scheme, which allows multiple parties to collaboratively generate search logs with boosted utility while satisfying differential privacy. To this end, a protocol, namely *CELS*, is proposed to meet the privacy-preserving objectives. Comparing with the above applications of HE, the unique feature of our approach is that it encrypts the model and apply HE re-encryption to record the model's status change. Our approach facilitates online model checking and verification, without revealing its secret. Furthermore, the encrypted model can be deployed on a platform whose TCB has not been authenticated. Although HE shows the benefits of privacy and secrecy guarantees, researchers also show that both HE schemes are prohibitively slow and has only supported limited crypto operations [24].

3 The Threat Model

To specify the threat model without loss of generality, we assume that the sandbox of the state verifier, which generates and maintains the secret keys (including HE's and OPE's) is secure. Given that the secret inside a sandbox could be leaked either from the VM [25], [26] or enclave [27], [28], [29] through the side-channel attacks, we also assume that the system is unbreachable during the process of the secret keys generation. In addition, the integrity of the pseudo-random number generator (PRNG) has been attested. Furthermore, we assume that once the mutual trust is established, the secret keys used for encryption are *ephemeral* and thus cannot be stolen by the inside attacker through the race condition [30] and the side-channel attack [31]. Finally, we assume that the information of e-Relations, which predominates the update of the secure model, should also be secured.

H-Verifier assumes that the cloud provider is *honest but curious* and does not impose any assumption on the trustworthiness of the state updater. Thus,

it is resilient against the exploits that breach the stand-alone sandbox of state updaters. For each sandbox, the adversary owns the capabilities of 1) stealing the ciphertext of the model; 2) running the brute force attack to break the ciphertext, and 3) eavesdropping the communication between the sandboxes. Our current design might still be vulnerable to the advanced collusion attack, in which the collusive adversary records the memory locations of the HE operations in all sandboxes and infers the rules of the transition function. In such a manner, the adversary can replace the ciphertext of the secure model with any arbitrary value, and thus break our system. The countermeasure of how to mitigate the collusion attack is discussed in Section 6.

4 System Design

In this section, we first present the system architecture of H-Verifier. Then, we describe the design of the encrypted model readily to be verified. After that, we detail the procedure and algorithms that initialize, update, and verify the encrypted model.

4.1 The System Architecture

H-Verifier is designed to identify the state of the monitored system/program against the specified formal model and encrypted system log. The primary challenge that H-Verifier intends to overcome is to facilitate trusted computing without solely relying on the guarantee of the TCB. It also prevents the plaintext of the system log from being recorded by the honest but curious cloud provider. To this end, H-Verifier is designed to fulfill three objectives: 1) it ensures the *confidentiality* of the formal model that specifies the intended behavior of the monitored system/program. The secrecy of the model does not only depends on the hardness of the sandbox or enclave, but also the cryptographic scheme used to generate the model; 2) it ensures that the state of system/program is verifiable during the runtime, without leaking the secret that how the model is verified; 3) it is resilient against the replay attack [32] and the side channel attacks [25], [27], even when the infrastructure is compromised. The bottom line is that H-verifier keeps the secrecy of both the system log and the formal model.

To achieve these objectives, we design H-Verifier as illustrated in Figure 1, which consists of three major components: *Event Generater*, *State Verifier*[2], and *State Updater*. To separate the functionalities, the components can be placed inside either the *heavy-weight* sandboxes such as the virtual machines (VMs), or the *light-weight* ones such as the containers and enclaves. H-Verifier works at two phases: the *auditing* and the *verification* phase. During the auditing phase, H-Verifier updates the system model upon the observed system events. Specifically, the event generator generates the raw system log e and timestamp t in plaintext

[2] In the real application, the *Event Generater* and *State Verifier* can be deployed in the same sandbox.

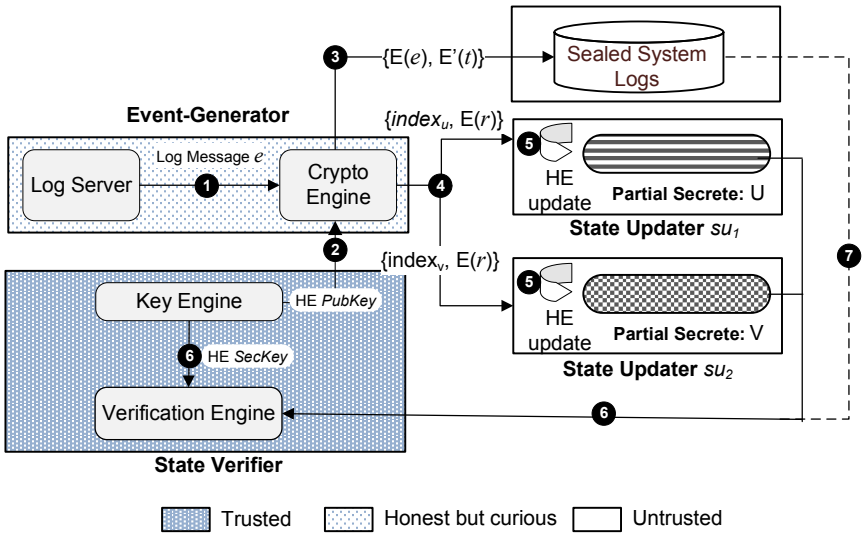


Fig. 1: The system architecture of *H-Verifier*.

(step 1), encrypts them with the HE public key, *PubKey*, which is generated by the state verifier (step 2), and seals them into the system log repository for future verification purpose (step 3). Rather than sending the data in plaintext across the boundary of the sandbox, the plaintext data are encrypted by the HE and Order-Preserving Encryption (OPE) cipher (E and E') [33] respectively, which produce a ciphertext pair $\{E(e), E'(t)\}$ (step 3). This process ensures that neither the key-related information nor the plaintext log will leave the sandbox unencrypted. Then, the tuples $\{index_u, E(r)\}$ and $\{index_v, E(r)\}$ are sent to two state updaters (su_1 and su_2), respectively (step 4). $E(r)$ is the HE ciphertext of a pseudo-random number r. The detail description of $index_u$, $index_v$, and $E(r)$ will be presented in Section 4.3. When the tuple reaches to each state updaters, it will be recomputed with the security model, which is also in the form of HE ciphertext but has been partitioned into two parts, namely U and V (step 5). The details of how to partition and update the model are also described in Section 4.3. During the verification phase, H-Verifier identifies the current state of the system being monitored. Specifically, the state verifier first retrieves the HE secret key *SecKey* from the local key repository. Then, it acquires the ciphertext of U and V from the state updaters (step 6). Since both U and V are encrypted by HE cipher, they are only decipherable inside the state verifier. Optionally, the sealed log might be fed into the state verifier for the offline verification (step 7).

H-Verifier is designed to demonstrate at least four advantages. First, it prevents the single-point failure caused by the compromised node. Since the jobs of the event generation, system auditing, and state verification are separated,

the security of H-Verifier primarily relies on the trustworthiness of state verifier, which keeps the secret keys confidentially. Second, the model never saves as plaintext in sandboxes, which preserves the secret of the model from being stolen by the malicious insider or the honest but curious privileged user. The sandboxes of state updater do not store any secret key, which also prevents the leakage of secret key caused by the side-channel attacks [25], [27]. Third, although the encrypted model is updated by recomputing with the ciphertext of the event log, the rules used to verify the system state is decoupled from the model, and thus do not depend upon the updated model. In the worst case, even the ciphertext of the model can be breached by the adversary, she still cannot figure out the state of the model because both the initial values of the elements in the model and the updating ciphertext are randomly chosen. Last but not least, instead of keeping the model as a monolithic piece, the model is partitioned into n pieces ($n \geq 2$), which are updated in an individual sandbox without any dependency. The HE re-computing against the partitioned model mitigates the runtime overhead caused by solving the large polynomial of HE. To increase the degree of obfuscation, we discuss the measure of how to partition the formal model into n ($n >= 2$) parts in Section 6.

4.2 Constructing Secure Model

To identify the state of the monitored system/program with both the encrypted formal model and the encrypted system log, at least two challenges are needed to be overcome. First, we need to formalize the model in a way that it is expressive enough to represent the general system state based on observations. Second, the model should be encrypted in a way that it will be difficult for the adversary to learn any criteria how the state of the monitored system is determined. To overcome these challenges, we first formalize the model as the finite state machine (FSM), which has been extensively applied as an efficient model to verify the system state. It has been adopted in various fields, such as modeling verification [34] and intrusion detection [35].

Definition 1 (The Security Model $\mathcal{M}$). The model used to identify the verifiable state of a cloud service can be defined as a deterministic finite state machine (FSM) $\mathcal{M}$, which is a quintuple $\mathcal{M} = (\Sigma, S, s_0, \Delta, F)$:

- Σ is an *event alphabet* with a finite number of symbols. In our system, each symbol e is an observable audit event generated by a cloud service.
- S is a finite set of the states.
- s_0 is the initial state, where $s_0 \in S$.
- Δ is a state-transition function: $\Delta : S \times \Sigma \rightarrow S$.
- F is the set of final states. In accordance with the general definition of FSM, we include an additional state $s_e \in F$, which is used to indicate an erroneous final state that leads the FSM to halt with errors.

As an important design objective, once the model is generated, it should be encrypted in a way that it is difficult for the adversary to learn how the state

of the monitored system is determined. Our methodology is based on the key observation: if we can successfully conceal 1) the information of the alphabet Σ in the transition function and 2) the correlation between current state S_c and the next state S_n in one transition, we will prevent the adversary from inferring the state of a system from the encrypted model. In order to conceal the information of the transition function, we decouple the actions of FSM update and verification. In particular, the FSM update refers to the actions that apply the HE re-encryption against the partitioned model upon new observations; whereas the FSM verification refers to the actions that apply the HE re-encryption and decryption to cancel out the intermediate states recorded by the partition and therefore determine the final state of the FSM.

To generate an encrypted model that can be easily updated and verified, we further refine the problem by answering three questions: 1) *how to partition and encrypt the model in a way that the partial model alone cannot be used to infer the entire model?* 2) *how to protect the secret of the transition function, even when the content of partial model is disclosed to the adversary?* and 3) *how to bound the overall computational overhead of model update and verification, with respect to the HE operations imposed to the model?* To answer the first question, we define a data structure for an FSM $\mathcal{M}$, namely *dual-vector* $DV(\mathcal{M})$ [3], which is used to keep the state information of an FSM. The definition of $DV(\mathcal{M})$ is listed below:

Definition 2 (The Dual-Vector $DV(\mathcal{M})$). An $DV(\mathcal{M})$ comprises two vectors of finite sizes, namely $U[sizeof(U)]$ and $V[sizeof(V)]$, where the functions $sizeof(U)$ and $sizeof(V)$ return the sizes of vectors U and V respectively. The initial values of the element are first randomly chosen and then encrypted with the HE public key.

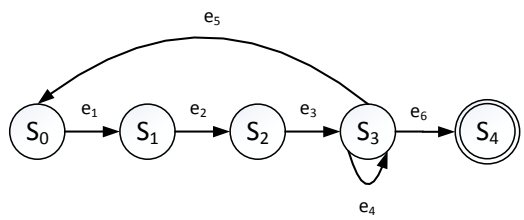
To answer the second question, we define two types of relations, namely *e-Relations* and *i-Relations*, which can be used as the alternatives to the original transition function of an FSM for the state update and verification purposes. Instead of keeping the transition function confidential, we use the information kept in the *e-Relations* to update the FSM based on the observable events, while we use the information kept in the *i-Relations* to verify the state of the FSM. The definitions of *e-Relations* and *i-Relations* are listed below:

Definition 3 (*e-Relations* and *i-Relations*). Given an FSM $\mathcal{M}$ and its transition function $\mathcal{M}.\Delta$, for each transition $\delta_i \in \mathcal{M}.\Delta$, we define a mapping relationship $\mathbb{M} : \delta_i \rightarrow \{index_u^i, index_v^i\}$, in which $index_\alpha^i$ ($\alpha = u$ or v) refers to the index corresponding to the vectors U and V, respectively.

- The *explicit relations* (*e-Relations*) is an ordered pair $E(i) = \{index_u^i, index_v^i\}$, where the states S_i and S_{i+1} satisfy the transition function δ_i: $S_i \times e_i \rightarrow S_{i+1}$.
- The *implicit relations* (*i-Relations*) is an ordered pair $I(i) = \{index_v^i, index_u^{i+1}\}$, where the transition δ_i: $S_i \times e_i \rightarrow S_{i+1}$ is corresponding to the e-Relation

[3] Our approach allows more than two vectors to be used, though the current design only uses two vectors.

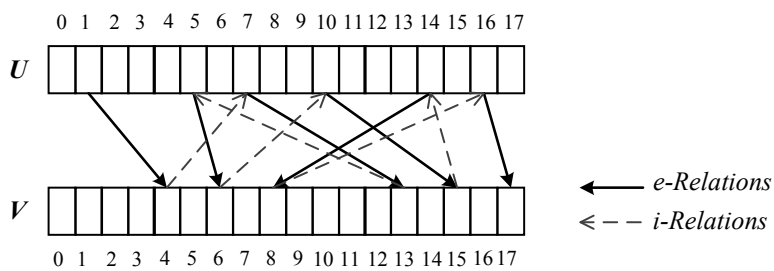
$\{index_u^i, index_v^i\}$ and the transition δ_{i+1}: $S_{i+1} \times e_j \rightarrow S_{i+2}$ is corresponding to the e-Relation $\{index_u^{i+1}, index_v^{i+1}\}$.



(a) The sample finite state machine $\mathcal{M}_s$.

Trans.	U	V
δ_0	1	4
δ_1	7	13
δ_2	5	6
δ_3	10	15
δ_4	14	8
δ_5	16	17

(b) The transition function $\mathcal{M}_s.\Delta$ and its corresponding e-Relations.



(c) The dual-vector DV($\mathcal{M}_s$)

Fig. 2: An example FSM and its corresponding dual vector.

To explain the concepts of *e-Relations* and *i-Relations*, we use the sample FSM illustrated in Figure 2.

Example 1. As illustrated in Figure 2a, the sample FSM $\mathcal{M}_s$ comprises an event alphabet $\Sigma = \{e_i\}$ $(0 \leq i \leq 5)$, the set of states $S = S_i$ $(0 \leq i \leq 4)$, the initial state S_0, and the final state set $F = \{S_4\}$. In Figure 2b, each row in the table represents an e-Relation. Accordingly, the solid arrows point from the elements of vector U to those of V are the e-Relations, while the dotted arrows point from the elements of vector V to those of U are the i-Relation. In this example, the transitions $\delta_1 : S_0 \times e_0 \rightarrow S_1$ (shown in the row 2) is mapped to the index pair $\{1, 4\}$, whose corresponding e-Relation (shown as the solid arrow) points from $U[1]$ to $V[4]$. Similarly, the transition $\delta_2 : S_1 \times e_1 \rightarrow S_2$ (the row 3) is mapped to the e-Relation points from $U[7]$ to $V[13]$. Thus, the i-Relation that corresponds to δ_1 and δ_2 is $\{4, 7\}$, which is the dotted arrow points from $V[4]$ to $U[7]$.

4.3 Updating Secure Model

A major objective of partitioning the model is to ensure that each partition is updated independently, and thus mitigates the chance of correlation. To achieve

this objective, the vectors U and V are deployed in different sandboxes and updated independently upon the observed events. However, to update the partitions in the sandboxes of the state updaters is non-trivial because U and V only keep the HE ciphertext as their elements, they only accept the HE ciphertext, in order to update the model in the state updaters.

Algorithm *Update_Partitions.*

Input: The pair $P_i = \{index_\alpha, E(r)\}$ ($\alpha = u$ or v) and the current vectors U and V.
Output: The updated vectors U' and V'.
1: **if** the state updater contains U **then**
2: $Sub(U[index_u], E(r))$;
3: **else**
4: $Add(V[index_v], E(r))$;
End of Algorithm

To do this, we apply the following conversion for each plaintext event e_i: for the transition δ_i: $S_i \times e_i \rightarrow S_{i+1}$ that takes e_i as an input, the event generator produces two pairs, namely $P_i = \{index_\alpha, E(r)\}$ ($\alpha = u$ or v), and send them to the sandbox that hosts U and V respectively. In the pairs, $index_\alpha$ ($\alpha = u$ or v) refers to the indexes in an e-Relation $E(i) = \{index_u^i, index_v^i\}$, while $E(r)$ is the ciphertext of a pseudorandom number r that is encrypted by HE public key $PubKey$. Algorithm *Update_Partitions* presents the procedure of updating vectors U and V upon the observed events. This algorithm uses two standard HE functions: namely *Add* for HE addition and *Sub* for HE subtraction. The algorithm is executed each time when a data pair is received by the state updater. For the sandbox that contains U, the element $U[index_u]$ will be *homomorphically* subtracted by $E(r)$. Otherwise, the element $V[index_v]$ will be *homomorphically* added by $E(r)$. Of course, the values of index $index_\alpha$ must also be encrypted to keep its confidentiality. Since Algorithm *Update_Partitions* ensures that each transition of $\mathcal{M}_s$ can be translated into an HE operations (either addition or subtraction) and imposed to each vector, we answer the third question mentioned in Section 4.3 that the time complexity of algorithm *Update_Partitions* is $O(1)$.

Whenever there is a need of checking the current state of the monitored program with respect to its corresponding FSM, the algorithm *Verification_State* is executed, which takes current values of U and V, as the input. The values of U and V might have been updated before verification. It outputs the ID of the state as defined by the FSM if success, or -1 if fail. To understand the algorithm, two standard HE functions are needed for HE encryption and HE decryption, namely *En* for *De*. Two additional functions *Lookup_expl* and *Lookup_impl* are needed to retrieve the indexes of the explicit and the implicit relations, respectively. For the verification purpose, the original copies of U and V, namely V_{part}^{org} and U_{part}^{org}, are also needed. The algorithm contains two parts. The lines 1 - 8 cancel out the states, which have been involved in the state transition during in

run-time, but are included in the set of final states. The line 10 - 15 traverse the transition function and determine which state is the final state.

Algorithm *Verification_State.*

Input: The updated vectors U'_{part} and V'_{part}, the original copy of partitions U^{org}_{part} and V^{org}_{part}, and the pseudorandom number r.

Output: The ID of the current state of the monitored system S_m if success, or -1 if fail. LineComment*Cancel out the non-final states.*

1: **for each** $\delta_i \in \Delta$
2: $(index_v, index_u) \leftarrow Lookup_imp\ (i)$;
3: $temp \leftarrow Add(EV[index_v], EV[index_u])$;
4: **if** $De(temp) == V[index_v] + U[index_u]$ **then**
5: $diff = De(EV[index_v]) - V[index_v]$;
6: $temp_V[index_v] \leftarrow Sub(EV[index_v], En(diff))$;
7: $temp_U[index_u] \leftarrow Add(EU[index_u], En(diff))$;
8: ▷ *Now, start to traverse the transition function and determine the final state*
9: **for each** $\delta_i \in \Delta$
10: $(index_u, index_v) \leftarrow Lookup_exp\ (i)$;
11: **if** $(De(temp_U[index_u]) == U^{org}_{part}[index_u])$ **and** $(De(temp_V[index_v]) == V^{org}_{part}[index_v] + r)$ **then**
12: return i;
13: **if** $De(temp) == V[index_v] + U[index_u]$ **then**
14: return -1;

End of Algorithm

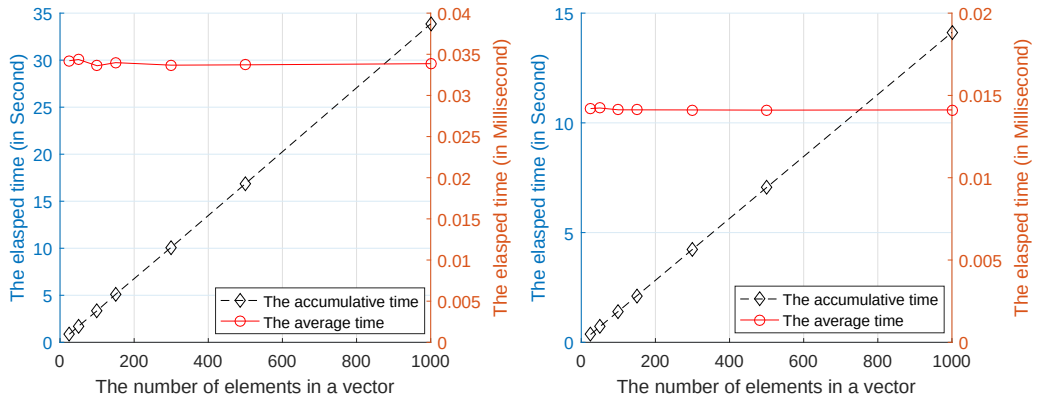
5 Implementation & Evaluation

We implemented the prototype of H-Verifier with nearly 500 lines of `C++` code. We used `HElib` [36] as the library to facilitate the generic HE operations. The evaluation results were collected from the physical machine which is equipped with Intel i7-6600U CPU 2.60GHz, 4-core processor, 20GB of RAM. In the following, we first analyze the computational overhead and then present the scalability of our model.

5.1 Computational Overhead

The first set of experiments measures the time elapsed for bootstrapping, updating, and verifying the model for different sizes of the vector, which is illustrated in Fig. 3 and Table 1. From the experiments, we have made the following observation: First, regardless of bootstrapping and verification, the average time spent for manipulating one element in a vector is nearly constant ($\sim 33ms$ for bootstrapping and $\sim 14ms$ for verification). Therefore, the accumulative time that bootstraps and verifies the vectors is linear to the size of a vector. Second, since the e-Relations is a pair of two indexes in U and V respectively, only one element

in each vector is updated for an observable event during the runtime. As shown in Table 1, the average time spent in each sandbox is nearly constant ($\sim 2.6ms$) regardless of the size of a vector. This property gives us the lower bound of the applications that can adapt our approach without losing the accountability. Third, the increasing size of the model, in particular, the increasing size of its transition function, requires the larger vectors to hold the HE ciphertext. However, the increasing size of its transition function is not necessarily linear to the size of vectors. Some dummy data are kept in the vectors for the purpose of obfuscation because only the elements at the indexes of the e-Relations and i-Relations contribute to the FSM updated and verified respectively. Similar results can be found in Fig. 3b for the state verification.



(a) The time elapsed for initialization (b) The time elapsed for verification

Fig. 3: The time elapsed for the incurred HE operations.

Table 1: The time elapsed for secure model update (in Millisecond).

	U	V
Mean	2.669	2.698
Std. Deviation	0.578	0.618

6 Discussion

In this section, we discuss some limitation of the proposed scheme, propose some possible solutions, and describe some directions for our future study.

First, although homomorphic encryption offers the good characteristics to compute over the ciphertext, its low efficiency prevents it from being applied to the high-performance applications [37], which seems to defeat the purpose of online state verification. We made the similar observation through our experiment. However, we found that the online component (the state updator) of H-Verifier only incurs one HE re-computation (either addition or subtraction) for one element in a vector, and its computational overhead is nearly constant. Meanwhile, the steps that consume more time, such as bootstrapping and verification, are performed offline. Therefore, our approach still holds the great promise to be adopted for various online auditing applications without losing the soundness. In our future study, we will test more real-world applications and study the performance gaps.

Second, our current scheme does not resilient against the collusion attack, in which the adversary controls all the state updater and records all the HE operations. In particular, the adversary records the HE addition and subtraction in each sandbox, identifies the updated elements in each vector, and infers the e-Relations. She can thus infer an i-Relations by correlating two adjacent e-Relations. To counter this attack, we can use the following scheme to increase the degree of randomness: First, instead of saving partitions in two vectors, we may use multiple vectors, say m ($m \geq 2$) vectors. To update a vector, a one-dimensional matrix of indexes (with an average dimension of n) will be sent to a state updater so that multiple elements, instead of one element in the vector, will be updated. However, during the verification process, only one pair of the index will be selected out of $m \times n$ possibilities. This approach will introduce the degree of randomness and mitigate the collusion attack. The only sacrifice is that it requires more sandboxes and will incur higher online overhead in each sandbox.

Third, in this paper, we primarily focus on retaining the secrecy of the model and without having to construct the TCB. Our current implementation requires the model to be generated as the results of the static analysis from the source code. The model that we have studied is deterministic, instead of probabilistic. As a future direction, we will apply the learning algorithms, such as Hidden Markov Model (HMM), to generate the non-deterministic model and consider the probabilistic model as well. Moreover, we will consider the side channel attack and the forensic analysis, which observe the CPU usage of H-Verifier and infer the execution path of the monitored system.

7 Conclusions

Despite its appealing features, such as ubiquity, elasticity, and low-cost, the cloud computing is still hindered by the limitation of real-time verification, detection, and accountability. Cloud providers intended to facilitate trusted computing with the supporting hardware and software, which lacks interoperability and the cross-platform security guarantee. In this paper, we present an online system, namely *H-Verifier*, which verifies the status of the confidential system

with collaborative sandboxes. To ensure data confidentiality, H-Verifier leverages homomorphic encryption (HE), partitions the model of the monitored system, and stores the partition model distributively. To track the system status, the partitioned model is updated against the ciphertext through the general HE operations. To keep both the model and events encrypted, H-Verifier does not require any TCB to be constructed on the computational nodes. Only a limited number of HE operations are needed to track the status of an online system. H-Verifier does not rely on any special hardware and thus can be widely deployed in a variety of environment. The evaluation demonstrates that H-Verifier can achieve reasonable performance overhead for model initialization, updating, and verification for an online system in the cloud.

Acknowledgment

This work is partially supported by the National Science Foundation under awards Grants No. DGE-1723707 and the Michigan Space Grant Consortium. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the funding agencies. We also thank Aditi Patil for the preliminary implementation.

References

1. M. Darwish, A. Ouda, and L. Capretz, "Cloud-based DDoS attacks and defenses," in *Proc. 2013 Int. Conf. Info. Society*, 2013, pp. 67–71.
2. V. Ciancaglini, M. Balduzzi, R. McArdle, and M. Rösler, "Below the surface: Exploring the deep web," 2015, [Online]. Available: https://www.trendmicro.com/cloud-content/us/pdfs/security-intelligence/white-papers/wp_below_the_surface.pdf.
3. Symantec, "Avoiding the hidden costs of the cloud," [Online]. Available: https://www.symantec.com/content/en/us/about/media/pdfs/b-state-of-cloud-global-results-2013.en-us.pdf.
4. R. Samani and F. Paget, "Cybercrime exposed: Cybercrime-as-a-servic," 2013, [Online]. Available: http://www.mcafee.com/jp/resources/white-papers/wp-cybercrime-exposed.pdf.
5. D. Goodin, "Zeusbot found using Amazon's EC2 as C&C server," [Online]. Available: http://www.theregister.co.uk/2009/12/09/amazon_ec2_bot_control_channel/.
6. M. D. Ryan, "Cloud computing security: The scientific challenge, and a survey of solutions," *J. Syst. and Software*, vol. 86, no. 9, pp. 2263–2268.
7. C. S. Alliance, "CSA Security, Trust & Assurance Registry (STAR)," [Online]. Available: https://cloudsecurityalliance.org/star/#_overview.
8. N. Santos, R. Rodrigues, K. P. Gummadi, and S. Saroiu, "Policy-sealed data: A new abstraction for building trusted cloud services," in *Proc. 21st USENIX Secur. Symp*, 2012, pp. 175–188.
9. E. G. Sirer, W. de Bruijn, P. Reynolds, A. Shieh, K. Walsh, D. Williams, and F. B. Schneider, "Logical attestation: An authorization architecture for trustworthy computing," in *Proc. 23rd ACM Symp. Oper. Syst. Principles*, 2011, pp. 249–264.

10. S. Butt, H. A. Lagar-Cavilla, A. Srivastava, and V. Ganapathy, "Self-service cloud computing," in *Proc. 2012 ACM Conf. Comput. Comm. Secur.*, 2012, pp. 253–264.
11. F. Zhang, J. Chen, H. Chen, and B. Zang, "Cloudvisor: Retrofitting protection of virtual machines in multi-tenant cloud with nested virtualization," in *Proc. 23rd ACM Symp. Oper. Syst. Principles*, 2011, pp. 203–216.
12. R. K. Ko, P. Jagadpramana, M. Mowbray, S. Pearson, M. Kirchberg, Q. Liang, and B. S. Lee, "Trustcloud: A framework for accountability and trust in cloud computing," in *Proc. 2011 IEEE World Cong. Serv.*, 2011, pp. 584–588.
13. Intel, "Intel. 64 and IA-32 architectures software developer's manual," Tech. Rep., [Online]. Available: http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html.
14. Intel Corporation, "Intel trusted execution technology: Software development guide," Tech. Rep., [Online]. Available: http://download.intel.com/technology/security/downloads/315168.pdf.
15. F. McKeen, I. Alexandrovich, I. Anati, D. Caspi, S. Johnson, R. Leslie-Hurd, and C. Rozas, "Intel software guard extensions support for dynamic memory management inside an enclave," in *Proc. Hardw. and Arch. Supp. Secur. Privacy*, 2016, pp. 101–109.
16. B. Hay and K. Nance, "Forensics examination of volatile system data using virtual introspection," *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 3, pp. 74–82, Apr. 2008.
17. Y. Fu and Z. Lin, "Space traveling across vm: Automatically bridging the semantic gap in virtual machine introspection via online kernel data redirection," in *2012 IEEE Symposium on Security and Privacy*, May 2012, pp. 586–600.
18. T. Garfinkel and M. Rosenblum, "A virtual machine introspection based architecture for intrusion detection," in *In Proc. Network and Distributed Systems Security Symposium*, 2003, pp. 191–206.
19. Y. Fu and Z. Lin, "Bridging the semantic gap in virtual machine introspection via online kernel data redirection," *ACM Trans. Inf. Syst. Secur.*, vol. 16, no. 2, pp. 7:1–7:29, Sep. 2013. [Online]. Available: http://doi.acm.org/10.1145/2505124
20. C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the Forty-first Annual ACM Symposium on Theory of Computing*, ser. STOC '09. New York, NY, USA: ACM, 2009, pp. 169–178. [Online]. Available: http://doi.acm.org/10.1145/1536414.1536440
21. M. Hirt and K. Sako, "Efficient receipt-free voting based on homomorphic encryption," in *Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding*, ser. Lecture Notes in Computer Science, vol. 1807. Springer, 2000, pp. 539–556.
22. F. Li, B. Luo, and P. Liu, "Secure information aggregation for smart grids using homomorphic encryption," *2010 First IEEE International Conference on Smart Grid Communications*, pp. 327–332, 2010.
23. Y. Hong, J. Vaidya, H. Lu, P. Karras, and S. Goel, "Collaborative search log sanitization: Toward differential privacy and boosted utility," *IEEE Trans. Dependable Sec. Comput.*, vol. 12, no. 5, pp. 504–518, 2015. [Online]. Available: https://doi.org/10.1109/TDSC.2014.2369034
24. Z. Brakerski and V. Vaikuntanathan, "Efficient fully homomorphic encryption from (standard) lwe," in *Proceedings of the 2011 IEEE 52Nd Annual Symposium on Foundations of Computer Science*, ser. FOCS '11. Washington, DC, USA:

IEEE Computer Society, 2011, pp. 97–106. [Online]. Available: http://dx.doi.org/10.1109/FOCS.2011.12

25. Y. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Cross-vm side channels and their use to extract private keys," in *Proc. 2012 ACM Conf. Comput. Comm. Secur.* New York, NY, USA: ACM, 2012, pp. 305–316.
26. Y. Yarom and K. Falkner, "Flush+reload: A high resolution, low noise, l3 cache side-channel attack," in *23rd USENIX Security Symposium (USENIX Security 14)*. San Diego, CA: USENIX Association, 2014, pp. 719–732. [Online]. Available: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom
27. S. Lee, M.-W. Shih, P. Gera, T. Kim, H. Kim, and M. Peinado, "Inferring fine-grained control flow inside SGX enclaves with branch shadowing," in *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, 2017, pp. 557–574. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/lee-sangho
28. F. Brasser, U. Müller, A. Dmitrienko, K. Kostiainen, S. Capkun, and A.-R. Sadeghi, "Software grand exposure: SGX cache attacks are practical," in *11th USENIX Workshop on Offensive Technologies (WOOT 17)*. Vancouver, BC: USENIX Association, 2017. [Online]. Available: https://www.usenix.org/conference/woot17/workshop-program/presentation/brasser
29. R. Guanciale, H. Nemati, C. Baumann, and M. Dam, "Cache storage channels: Alias-driven attacks and verified countermeasures," in *2016 IEEE Symposium on Security and Privacy (SP)*, vol. 00, May 2016, pp. 38–55. [Online]. Available: doi.ieeecomputersociety.org/10.1109/SP.2016.11
30. D. Dean and A. J. Hu, "Fixing races for fun and profit: How to use access(2)," in *Proc. 13th USENIX Secur. Symp.*, 2004, pp. 14–14.
31. M. Schwarz, S. Weiser, D. Gruss, C. Maurice, and S. Mangard, "Malware guard extension: Using SGX to conceal cache attacks," *CoRR*, vol. abs/1702.08719, 2017.
32. G. Dua, N. Gautam, D. Sharma, and A. Arora, "Replay attack prevention in Kerberos authentication protocol using triple password," *CoRR*, vol. abs/1304.3550, 2013.
33. A. Boldyreva, N. Chenette, Y. Lee, and A. O'Neill, "Order-preserving symmetric encryption," in *Proceedings of the 28th Annual International Conference on Advances in Cryptology: The Theory and Applications of Cryptographic Techniques*, ser. EUROCRYPT '09. Berlin, Heidelberg: Springer-Verlag, 2009, pp. 224–241. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-01001-9_13
34. D. Lee and M. Yannakakis, "Principles and methods of testing finite state machines-a survey," *Proceedings of the IEEE*, vol. 84, no. 8, pp. 1090–1123, Aug 1996.
35. S. Bhatkar, A. Chaturvedi, and R. Sekar, "Dataflow anomaly detection," in *2006 IEEE Symposium on Security and Privacy (S&P 2006), 21-24 May 2006, Berkeley, California, USA*, 2006, pp. 48–62. [Online]. Available: https://doi.org/10.1109/SP.2006.12
36. HElib: the library that implements homomorphic encryption (HE), [Online]. Available: https://github.com/shaih/HElib.
37. C. Gentry, "A fully homomorphic encryption scheme," Ph.D. dissertation, Stanford, CA, USA, 2009, aAI3382729.